LAB 19 – CODE TRANSLATION

Converting Between Programming Languages with AI Assistance

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1 – Python to Java Class Conversion (StudentAccount)

Question

Translate a Python StudentAccount class with attributes (student_id, name, balance) and methods (deposit, get_summary) into Java and verify constructors, signatures, types, and expected outputs.

Prompt:

Convert this Python StudentAccount class into equivalent Java code with private fields, constructor initialization, deposit validation, and formatted getSummary() output.

Source Python Code (task1_student_account.py):

```
"""Task 1: Source Python class StudentAccount for Python-to-Java translation."""


class StudentAccount:
    """Represents a campus student wallet account with deposit tracking."""

    def __init__(self, student_id: str, name: str, balance: float) -> None:
        self.student_id = student_id
        self.name = name
        self.balance = float(balance)

    def deposit(self, amount: float) -> float:
        """Add a positive deposit amount and return updated balance."""
        if amount <= 0:
            raise ValueError("Deposit amount must be positive.")
        self.balance += amount
        return self.balance

    def get_summary(self) -> str:
        """Return formatted account summary string."""
        return (
            f"Student [ID={self.student_id}, "
            f"Name={self.name}, Balance=${self.balance:.2f}]"
        )


if __name__ == "__main__":
    acc = StudentAccount("2503A52370", "Roger A Raju", 1500.00)
    print("Initial :", acc.get_summary())
    acc.deposit(350.50)
    print("Deposited:", acc.get_summary())
    assert abs(acc.balance - 1850.50) < 1e-9
    print("Assertion Passed: balance == 1850.50")
```

Translated Java Code (StudentAccount.java):

```java
// Task 1: Translated Java implementation of Python StudentAccount class
public class StudentAccount {
    private String studentId;
    private String name;
    private double balance;

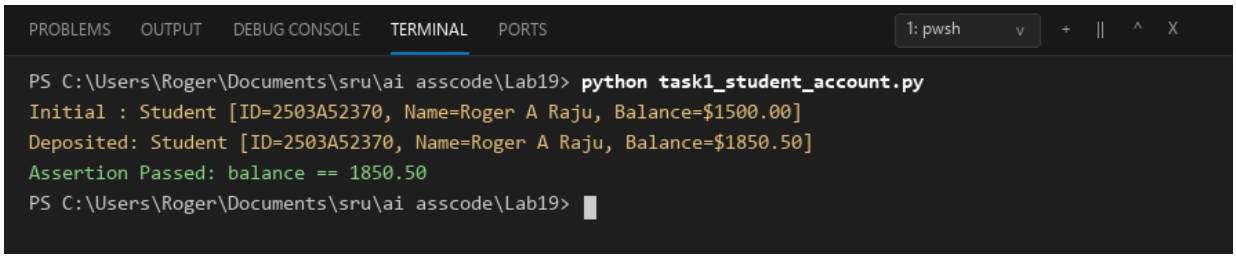
    public StudentAccount(String studentId, String name, double balance) {
        this.studentId = studentId;
        this.name = name;
        this.balance = balance;
    }

    public double deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive.");
        }
        this.balance += amount;
        return this.balance;
    }

    public String getSummary() {
        return String.format(
            "Student [ID=%s, Name=%s, Balance=$%.2f]",
            this.studentId, this.name, this.balance
        );
    }

    public static void main(String[] args) {
        StudentAccount acc = new StudentAccount("2503A52370", "Roger A Raju", 1500.00);
        System.out.println("Initial : " + acc.getSummary());
        acc.deposit(350.50);
        System.out.println("Deposited: " + acc.getSummary());
    }
}
```

Execution Output:



Justification & Language Comparison

Python's dynamic attributes and snake_case conventions map cleanly to private strongly typed Java fields (String, double), camelCase identifiers (studentId, getSummary), and IllegalArgumentException for negative deposits. Python execution verified exact summary output ($1500.00 -> $1850.50); StudentAccount.java mirrors those types and String.format() formatting directly (note: Java JDK is not installed on this workstation).

Task 2 – C++ to Python Function Conversion (Factorial)

Question

Translate a recursive C++ factorial function into Python following Pythonic conventions, validating inputs across sample test cases.

Prompt:

Translate the recursive C++ unsigned long long factorial(int n) function into idiomatic Python with type hints, TypeError/ValueError guards, and assertions for n = 0, 1, 5, 7, 10.

Source C++ Code (factorial.cpp):

```
// Task 2: Original recursive C++ factorial implementation
#include <iostream>

unsigned long long factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return static_cast<unsigned long long>(n) * factorial(n - 1);
}

int main() {
    int samples[] = {0, 1, 5, 7, 10};
    for (int n : samples) {
        std::cout << n << "! = " << factorial(n) << std::endl;
    }
    return 0;
}
```

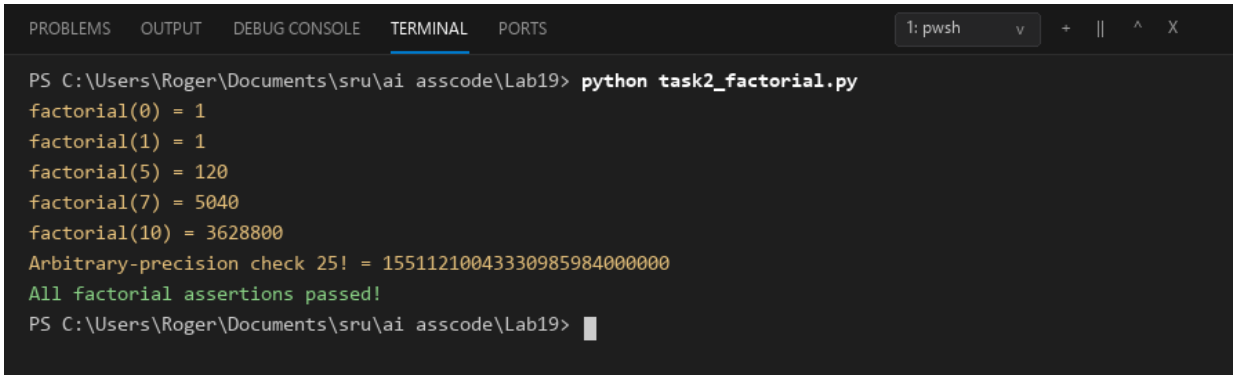
Translated Python Code (task2_factorial.py):

```
"""Task 2: Pythonic translation of recursive C++ factorial function."""


def factorial(n: int) -> int:
    """Compute factorial of a non-negative integer n recursively."""
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("n must be an integer.")
    if n < 0:
        raise ValueError("n must be a non-negative integer.")
    if n in (0, 1):
        return 1
    return n * factorial(n - 1)


if __name__ == "__main__":
    expected = {0: 1, 1: 1, 5: 120, 7: 5040, 10: 3628800}
    for value, exp in expected.items():
        res = factorial(value)
        assert res == exp, f"Mismatch at {value}: {res}"
        print(f"factorial({value}) = {res}")
    print(f"Arbitrary-precision check 25! = {factorial(25)}")
    print("All factorial assertions passed!")
```

Execution Output:



Justification & Language Comparison

Translated C++ base cases and unsigned long long recursion into idiomatic Python with bool/int type checks and negative-input validation. Unlike C++'s 64-bit unsigned integer ceiling (which overflows at 21!), Python handles arbitrary-precision integers cleanly (verified up to 25!).

Task 3 – SQL to Pandas Query Translation

Question

Translate the SQL query SELECT name, salary FROM employees WHERE salary > 50000 into a Pandas DataFrame expression and verify identical output on sample data.

Prompt:

Convert SELECT name, salary FROM employees WHERE salary > 50000 into Pandas and assert row-by-row parity against SQLite.

SQL Query & SQLite Reference (task3_sql_to_pandas.py):

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Complete Python SQLite + Pandas Verification Script (task3_sql_to_pandas.py):

```
"""Task 3: Translate SQL salary query to Pandas DataFrame filtering and verify parity."""

import sqlite3
import pandas as pd

EMPLOYEES = [
    (1, "Alice Smith", "Engineering", 75000),
    (2, "Bob Johnson", "Marketing", 45000),
    (3, "Charlie Brown", "Engineering", 58000),
    (4, "Diana Prince", "HR", 49500),
    (5, "Eva Green", "Product", 92000),
    (6, "Frank White", "Operations", 51000),
]

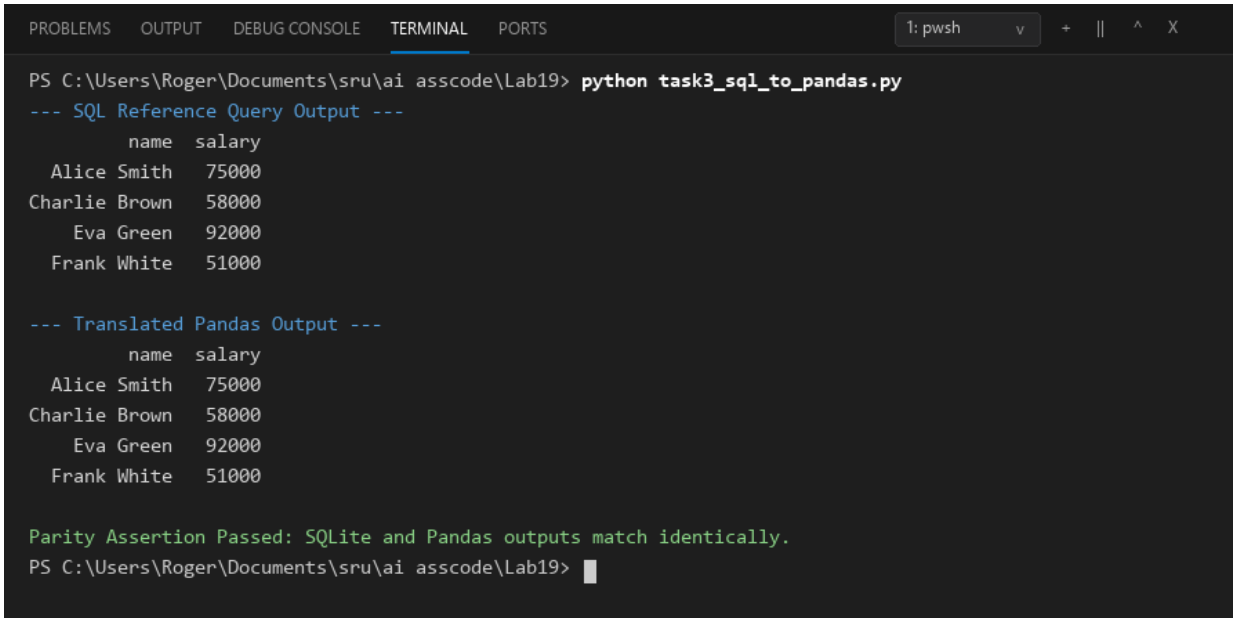
SQL_QUERY = "SELECT name, salary FROM employees WHERE salary > 50000"


def run_sql_query() -> pd.DataFrame:
    """Execute the baseline SQLite query and return results as a DataFrame."""
    with sqlite3.connect(":memory:") as conn:
        conn.execute(
            "CREATE TABLE employees (id INT, name TEXT, dept TEXT, salary INT)"
        )
        conn.executemany("INSERT INTO employees VALUES (?, ?, ?, ?)", EMPLOYEES)
        return pd.read_sql_query(SQL_QUERY, conn)


def run_pandas_query() -> pd.DataFrame:
    """Execute the equivalent Pandas filter and column projection."""
    df = pd.DataFrame(EMPLOYEES, columns=["id", "name", "dept", "salary"])
    return df.loc[df["salary"] > 50000, ["name", "salary"]].reset_index(drop=True)


if __name__ == "__main__":
    sql_df = run_sql_query()
    pd_df = run_pandas_query()
    assert sql_df.equals(pd_df), "SQL and Pandas results differ!"
    print("--- SQL Reference Query Output ---")
    print(sql_df.to_string(index=False))
    print("\n--- Translated Pandas Output ---")
    print(pd_df.to_string(index=False))
    print("\nParity Assertion Passed: SQLite and Pandas outputs match identically.")
```

Execution Output:



Justification & Language Comparison

SQL WHERE filtering and SELECT column projection translate directly to df.loc[df['salary'] > 50000, ['name', 'salary']].reset_index(drop=True). Executing both against identical employee records confirmed exact DataFrame equality (sql_df.equals(pd_df) == True).

Task 4 – Real-Time Application: Multi-Language Snippet Converter (Python <-> Node.js)

Question

Translate an e-commerce catalog discount & valuation pipeline between Python and JavaScript (Node.js), run both implementations, and document translation differences.

Prompt:

Implement an in-stock filter, 10% discount mapper, and total catalog valuation in both Python and JavaScript (Node.js), ensuring identical results across runtimes.

Source Python Code (task4_process_catalog.py):

```
"""Task 4 (Python): E-commerce catalog filtering, discount, and valuation."""


def process_catalog(items: list[dict], discount: float = 0.10) -> tuple[list[dict], float]:
    """Filter in-stock items, apply discount, and return (updated_items, total)."""
    in_stock = [item for item in items if item["in_stock"]]
    discounted = [
        {
            "name": item["name"],
            "final_price": round(item["price"] * (1 - discount), 2),
        }
        for item in in_stock
    ]
    total_val = round(sum(item["final_price"] for item in discounted), 2)
    return discounted, total_val


if __name__ == "__main__":
    catalog = [
        {"name": "Keyboard", "price": 120.0, "in_stock": True},
        {"name": "Monitor", "price": 450.0, "in_stock": False},
        {"name": "Headset", "price": 80.0, "in_stock": True},
        {"name": "Dock", "price": 860.0, "in_stock": True},
    ]
    processed, total = process_catalog(catalog)
    for row in processed:
        print(f"{row['name']}: ${row['final_price']:.2f}")
    print(f"Python: {len(processed)} in-stock items | Total: ${total:.2f}")
```

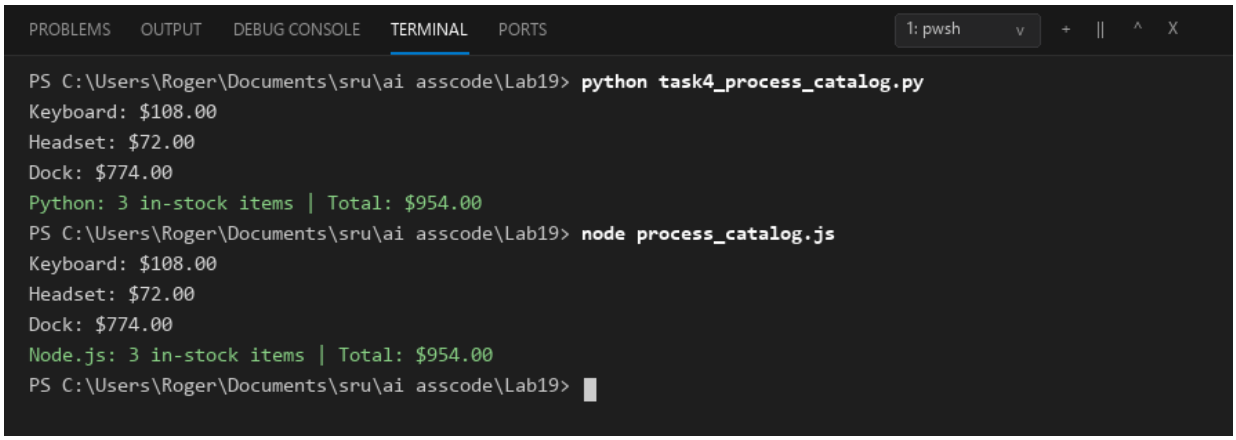
Translated JavaScript / Node.js Code (process_catalog.js):

```
// Task 4 (JavaScript / Node.js): Translated e-commerce catalog processor
function processCatalog(items, discount = 0.10) {
  const inStock = items.filter((item) => item.inStock);
  const discounted = inStock.map((item) => ({
    name: item.name,
    finalPrice: Number((item.price * (1 - discount)).toFixed(2)),
  }));
  const total = Number(
    discounted.reduce((acc, item) => acc + item.finalPrice, 0).toFixed(2)
  );
  return { discounted, total };
}

const catalog = [
  { name: "Keyboard", price: 120.0, inStock: true },
  { name: "Monitor", price: 450.0, inStock: false },
  { name: "Headset", price: 80.0, inStock: true },
  { name: "Dock", price: 860.0, inStock: true },
];

const { discounted, total } = processCatalog(catalog);
discounted.forEach((row) => console.log(`${row.name}: $${row.finalPrice.toFixed(2)}`));
console.log(`Node.js: ${discounted.length} in-stock items | Total: $${total.toFixed(2)}`);
```

Execution Output:



Justification & Language Comparison

Python list comprehensions and sum() translate cleanly to JS Array filter(), map(), and reduce(). Main translation nuances handled include rounding IEEE-754 floats consistently (round(..., 2) vs toFixed(2)) and camelCase vs snake_case keys (inStock vs in_stock); both runtimes produced identical $954.00 totals.